LeetCode Practice

9 Easy: Linked List

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

9
EASY

Patterns (in order)

1. Linked List (9)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

9 questions | tap a title to jump to it

Easy questions (9) Go >

Linked List (9) Go >

E 1. Merge Two Sorted Lists LC 21
E 2. Remove Duplicates from Sorted List LC 83
E 3. Linked List Cycle LC 141
E 4. Intersection of Two Linked Lists LC 160
E 5. Reverse Linked List LC 206
E 6. Palindrome Linked List LC 234
E 7. Design HashMap LC 706
E 8. Middle of the Linked List LC 876
E 9. Delete N Nodes After M Nodes of Linked List LC 1474

Easy questions (9)

EASY | TOPIC 1 OF 1

Linked List

9 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Merge Two Sorted Lists LC 21
2. Remove Duplicates from Sorted List LC 83
3. Linked List Cycle LC 141
4. Intersection of Two Linked Lists LC 160
5. Reverse Linked List LC 206
6. Palindrome Linked List LC 234
7. Design HashMap LC 706
8. Middle of the Linked List LC 876
9. Delete N Nodes After M Nodes of Linked List LC 1474

EASY | Linked List | LeetCode #21 | 1 of 9 | 1/9

Merge Two Sorted Lists

QUESTION

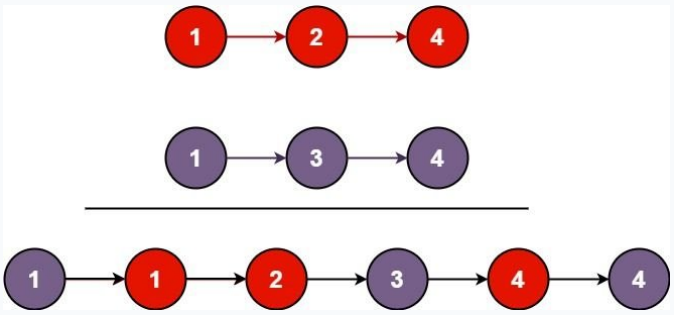
You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1

Input	`list1 = [1,2,4], list2 = [1,3,4]`
Output	`[1,1,2,3,4,4]`



Example 2

Input	`list1 = [], list2 = []`
Output	`[]`

Example 3

Input	`list1 = [], list2 = [0]`
Output	`[0]`

CONSTRAINTS

- The number of nodes in both lists is in the range [0, 50].
- 100 <= Node.val <= 100
- Both list1 and list2 are sorted in non-decreasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have two sorted linked lists and need to merge them into one sorted list

by re-linking the original nodes — no new allocations.

Return the new head. Right?"

"A few quick questions:

Can either list be empty? Yes — constraints say 0 nodes is valid.

Are duplicates across lists allowed? Yes.

Must the result be stable (preserve relative order of equal elements)? Yes, standard merge."

"Let me walk the example: list1=[1,2,4], list2=[1,3,4].

PHASE 1 CLARIFY

Pick 1 (tie -> list1 first), then 1 from list2, then 2, 3, 4, 4. -> [1,1,2,3,4,4] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = every value from both lists
sort values
build a new list from them
```

TIME	SPACE
O((n + m) log(n + m))	O(n + m)

WHY IT IS SLOW

Ignores that both lists are already sorted and allocates new nodes. Walking both lists with two pointers and relinking the smaller node is O(n + m) time and O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the sort — we paid O(log n) we didn't need.

Since both lists are already sorted, the smallest available element is always

at one of the two heads. We just pick the smaller head at each step.

Pseudocode:

`dummy = cur = ListNode(0)`

`while list1 and list2:`

`if list1.val <= list2.val: attach list1, advance list1`

`else: attach list2, advance list2`

advance cur

attach whichever list remains

`return dummy.next`

Time: O(m+n) — one comparison per node. Space: O(1) — only pointer rewiring.

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _21_MergeTwoSortedLists:
    def mergeTwoLists(self, list1, list2):
        dummy = cur = ListNode(0)
        while list1 and list2:
            if list1.val <= list2.val:
                cur.next = list1
                list1 = list1.next
            else:
                cur.next = list2
                list2 = list2.next
```

```
            cur = cur.next
        cur.next = list1 if list1 else list2
        return dummy.next
```

PHASE 5 TEST & DEBUG

"Let me trace mergeTwoLists([1,2,4],[1,3,4])."

dummy->curr. Compare heads.

1<=1 (take l1=1): curr.next=1. l1 advance to 2. curr=node(1).

l1=2 vs l2=1: 2>1 (take l2=1): curr.next=1. l2->3. curr=node(1).

l1=2 vs l2=3: 2<3 (take l1=2): curr.next=2. l1->4. curr=node(2).

l1=4 vs l2=3: 4>3 (take l2=3): curr.next=3. l2->4. curr=node(3).

l1=4 vs l2=4: equal (take l1=4): curr.next=4. l1->None. curr=node(4).

l1=None: append remaining l2=4. curr.next=l2.

Result: 1->1->2->3->4->4 ok

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m+n)	**O(1)**
one comparison per node	reuses existing nodes

The dummy head is the key trick: we never need to special-case the very first node.

Follow-up: Merge k Sorted Lists (#23) — use a min-heap for O(n log k) time.

Follow-up: Sort a Linked List (#148) — apply this same merge step inside merge sort.

EASY | Linked List | LeetCode #83 | 2 of 9 | 2/9

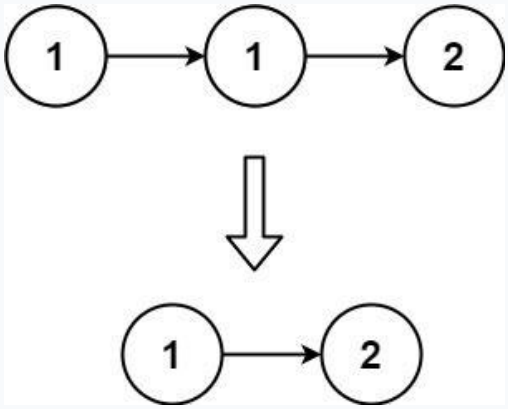
Remove Duplicates from Sorted List

QUESTION

Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

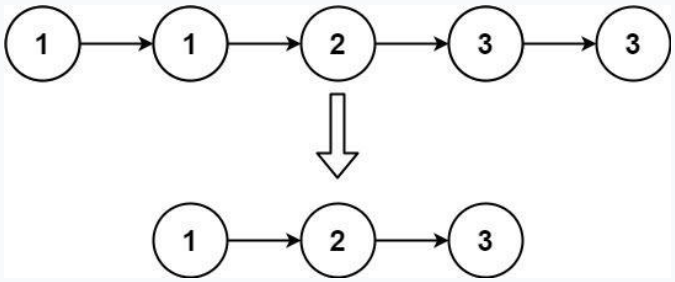
Example 1

Input	`head = [1,1,2]`
Output	`[1,2]`



Example 2

Input	`head = [1,1,2,3,3]`
Output	`[1,2,3]`



CONSTRAINTS

- The number of nodes in the list is in the range [0, 300].
- 100 <= Node.val <= 100
- The list is guaranteed to be sorted in ascending order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove all nodes with duplicate values — keep only ONE copy of each value.

List is sorted. Return modified head. Right?"

"A few quick questions:

Keep the first occurrence? Yes. Empty list -> None? Yes."

"Let me walk: [1,1,2] -> [1,2] ok. [1,1,2,3,3] -> [1,2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
seen = a set
keep each node only if its value is not in seen
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

The list is sorted, so duplicates are adjacent. Comparing each node with the next and bypassing equal ones is O(n) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single-pass in-place — O(n). Walk-through:"

`curr = head. Scan while curr and curr.next exist:`

If curr.val == curr.next.val: curr.next = curr.next.next (skip duplicate).

<- don't advance curr yet — next might also be a duplicate.

Else: curr = curr.next (values differ, safe to advance).

WHY don't advance when removing: after removing curr.next, the NEW curr.next

might equal curr.val too. Only advance when values differ.

Return head (head never changes — duplicates are interior or trailing).

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _83_RemoveDuplicatesSortedList:
    def deleteDuplicates(self, head):
        current = head
        while current and current.next:
            if current.val == current.next.val:
                current.next = current.next.next
            else:
                current = current.next
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace deleteDuplicates(1->1->2->3->3)."

curr=1. curr.next=1==curr: curr.next=curr.next.next=2. curr still at node(1).

curr.next=2!=1: curr=2. curr.next=3!=2: curr=3.

curr.next=3==3: curr.next=3.next=None. curr still at node(3). curr.next=None -> stop.

Result: 1->2->3 ok

PHASE 5 TEST & DEBUG

WHY don't advance curr when deleting: the new curr.next might also be a duplicate.

'1->1->2->3->3': stops at each new distinct value. Doesn't advance if deleted.

WHY not advance after delete: new curr.next might also be a duplicate of curr.val.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
visit each node once	only pointer manipulation

Follow-up: Remove Duplicates from Sorted List II (#82) — remove ALL nodes with duplicate values.

Needs a dummy head and tracks the previous non-duplicate node.

IN YOUR APP | Expose | Editorial | Community

EASY | Linked List | LeetCode #141 | 3 of 9 | 3/9

Linked List Cycle

QUESTION

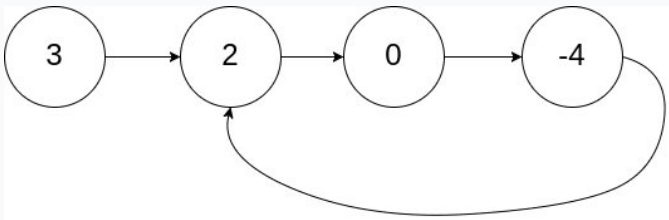
Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.

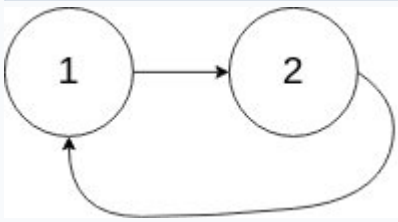
Example 1

Input	`head = [3,2,0,-4], pos = 1`
Output	`true`
Explanation	*There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).*



Example 2

Input	`head = [1,2], pos = 0`
Output	`true`
Explanation	*There is a cycle in the linked list, where the tail connects to the 0th node.*



Example 3

Input	`head = [1], pos = -1`
Output	`false`
Explanation	*There is no cycle in the linked list.*



CONSTRAINTS

- The number of the nodes in the list is in the range $[0, 10^4]$.
- 10^5 <= Node.val <= 10^5

QUESTION

- pos is -1 or a valid index in the linked-list.
- Follow up: Can you solve it using O(1) (i.e. constant) memory?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have a singly linked list and need to return true if there is a cycle —

meaning some node's next pointer points back to a previous node.

Return false if we reach None (no cycle). Right?"

"A few quick questions:

Is pos given to us in the actual function? No — pos is just for test descriptions.

Can the list have one node pointing to itself? Yes — that's a cycle too.

Can the list be empty? Yes — empty list has no cycle."

"Let me walk the example: [3,2,0,-4], tail connects back to index 1.

3->2->0->-4->2->… cycles forever -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
seen = an empty set
walk the list:
    if the node is already in seen:
        return True, else add it
    return False at the end of the list
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but remembers every node. Floyd's tortoise and hare (slow moves 1, fast moves 2; they meet iff there is a cycle) uses O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra space for the visited set.

Floyd's tortoise-and-hare: use two pointers — slow moves 1 step, fast moves 2.

If a cycle exists, fast will eventually lap slow and they'll meet inside the cycle.

If no cycle, fast reaches None first.

Pseudocode:

```
slow = fast = head
```

PHASE 3 OPTIMIZE

```
while fast and fast.next:
slow = slow.next; fast = fast.next.next
if slow is fast: return True
return False
```

Time: O(n). Space: O(1) — only two pointer variables.

Does that make sense before I code it?"

Key insight: Floyd's slow/fast pointers guarantee a meeting inside a cycle within O(n) steps — if there's no cycle, fast exits at None first.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _141_LinkedListCycle:
    def hasCycle(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                return True
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace hasCycle([3,2,0,-4] with cycle back to index 1)."

Floyd's tortoise and hare. slow=head, fast=head.

Step 1: slow=2, fast=0. Step 2: slow=0, fast=2. Step 3: slow=-4, fast=-4.

slow==fast -> cycle detected -> return True ok

[1,2]: fast=2, fast.next=None -> return False ok (no cycle).

WHY fast moves 2x: if cycle exists, fast laps slow within (cycle_len) steps.

If fast reaches None: no cycle. O(n) time, O(1) space (no visited set needed).

Edge: [1] no cycle: fast=1, fast.next=None -> return False ok

Edge: [1,2] no cycle: fast=2, fast.next=None -> return False ok

WHY can't just use visited set here: problem requires O(1) space -> Floyd's is canonical.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
fast covers the cycle in at most n steps after entering it	only two pointer variables

Hash-set approach is O(n) space — Floyd's is the canonical O(1) improvement.

Follow-up: Find Cycle Start (#142) — after they meet, reset slow to head,

advance both one step at a time; they meet again at the cycle entry.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: cycle length — once they meet, keep fast moving and count steps until it returns.

IN YOUR APP | Expose | Editorial | Community

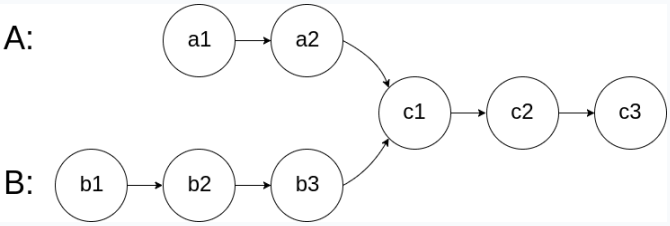
EASY | Linked List | LeetCode #160 | 4 of 9 | 4/9

Intersection of Two Linked Lists

QUESTION

Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1:



The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must retain their original structure after the function returns.

Custom Judge:

The inputs to the judge are given as follows (your program is not given these inputs):

CONSTRAINTS

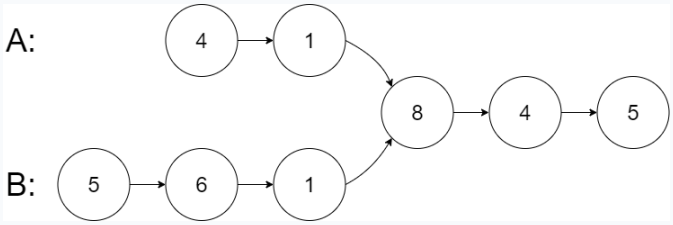
- intersectVal - The value of the node where the intersection occurs. This is 0 if there is no intersected node.
- listA - The first linked list.
- listB - The second linked list.
- skipA - The number of nodes to skip ahead in listA (starting from the head) to get to the intersected node.
- skipB - The number of nodes to skip ahead in listB (starting from the head) to get to the intersected node.

The judge will then create the linked structure based on these inputs and pass the two heads, headA and headB to your program. If you correctly return the intersected node, then your solution will be accepted.

QUESTION

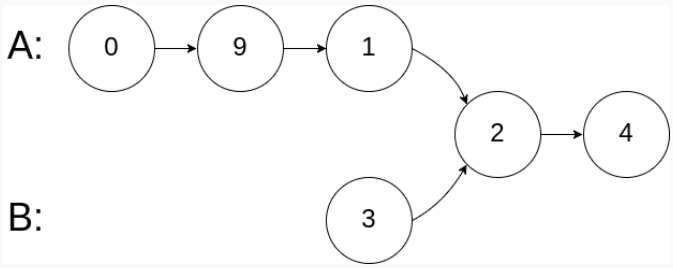
Example 1

Input	`intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3`
Output	`Intersected at '8'`
Explanation	*The intersected node's value is 8 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.* *- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.*



Example 2

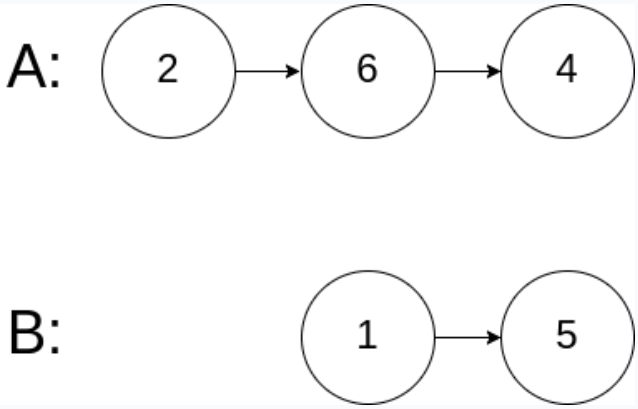
Input	`intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1`
Output	`Intersected at '2'`
Explanation	*The intersected node's value is 2 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [1,9,1,2,4]. From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.*



QUESTION

Example 3

Input	`intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2`
Output	`No intersection`
Explanation	*The two lists do not intersect, so return null.*



- The number of nodes of listA is in the m.
- The number of nodes of listB is in the n.
- 1 <= m, n <= 3 * 10^4
- 1 <= Node.val <= 10^5
- 0 <= skipA <= m
- 0 <= skipB <= n
- intersectVal is 0 if listA and listB do not intersect.
- intersectVal == listA[skipA] == listB[skipB] if listA and listB intersect.
- Follow up: Could you write a solution that runs in O(m + n) time and use only O(1) memory?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given heads of two singly linked lists, return the node where they intersect.

Return null if no intersection. By reference, not by value. Right?"

"A few quick questions:

Lists may intersect or not? Yes. No cycles? Correct."

"Let me walk: listA=[4,1,8,4,5], listB=[5,6,1,8,4,5] -> intersect at node with val=8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for a in as:
    # in list A
    walk all of list B
    if b is a:
        return it
return nothing
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

m * n node comparisons. Put list A's nodes in a hash set and walk B (O(m + n) time, O(m) space), or let two pointers swap to the other list's head at the end so they meet at the intersection in O(m + n) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra space for the set.

Two-pointer trick: walk both lists; when one reaches None, restart at the OTHER list's head.

Both pointers traverse len(A)+len(B) total steps and meet at the intersection (or both reach None).

Time: O(n+m). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _160_IntersectionLinkedLists:
    def getIntersectionNode(self, headA, headB):
        a, b = headA, headB
        while a is not b:
            a = a.next if a else headB
            b = b.next if b else headA
        return a
```

PHASE 5 TEST & DEBUG

"Let me trace getIntersectionNode where list A=[4,1,8,4,5], B=[5,6,1,8,4,5]."

Two pointers. pA=head_A, pB=head_B.

When pA reaches end, redirect to head_B. When pB reaches end, redirect to head_A.

After len(A)+len(B) steps, both have traversed total length. They meet at intersection.

A=4->1->8->4->5 (len 5). B=5->6->1->8->4->5 (len 6). Intersection at node(8).

pA path: 4,1,8,4,5,None->5,6,1,8. pB path: 5,6,1,8,4,5,None->4,1,8.

Both arrive at node(8) at step 11 ok

WHY redirect: equalizes path lengths. pA travels A+B = pB travels B+A. Same total distance.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(1)** only two pointer variables

This is LCA (Lowest Common Ancestor) of a Binary Tree (#236) applied to linked lists.

Follow-up: find the intersection VALUE (not node) — works even without node references.

Follow-up: if lists could have cycles, use Floyd's to detect cycles first.

IN YOUR APP | Expose | Editorial | Community

EASY | Linked List | LeetCode #206 | 5 of 9 | 5/9

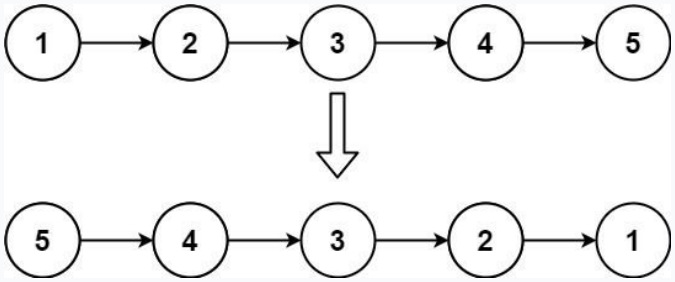
Reverse Linked List

QUESTION

Given the head of a singly linked list, reverse the list, and return the reversed list.

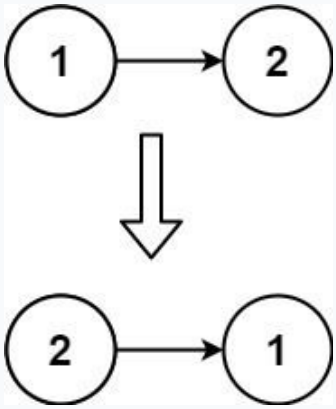
Example 1

Input	`head = [1,2,3,4,5]`
Output	`[5,4,3,2,1]`



Example 2

Input	`head = [1,2]`
Output	`[2,1]`



Example 3

Input	`head = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the list is the range [0, 5000].
- 5000 <= Node.val <= 5000
- Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given the head of a singly linked list, reverse it in-place and return the new head.

We change the next pointers — not just the values. Right?"

"A few quick questions:

Can the list be empty? Yes — return None.

Can it have just one node? Yes — return that node unchanged.

The follow-up asks for both iterative and recursive — I'll do iterative first."

"Let me walk the example: [1,2,3,4,5] -> [5,4,3,2,1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
read all values into an array
build a new list by pushing each value to the front
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Time is O(n), but it allocates a whole second list. Flipping each next pointer while walking (prev, current, next) reverses in place with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra space and the unnecessary node allocation.

We can reverse in-place by maintaining three pointers: prev, curr, next_node.

At each step we point curr.next backward to prev, then advance.

Pseudocode:

prev, curr = None, head

```
while curr:
next_node = curr.next
curr.next = prev
```

prev, curr = curr, next_node

```
return prev
```

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _206_ReverseLinkedList:
    def reverseList(self, head):
        prev, curr = None, head
        while curr:
            next_node = curr.next
            curr.next = prev
            prev, curr = curr, next_node
        return prev
```

PHASE 5 TEST & DEBUG

"Let me trace reverseList(1->2->3->4->5)."

Iterative: prev=None, curr=1.

curr=1: nxt=2. 1.next=None. prev=1. curr=2.

curr=2: nxt=3. 2.next=1. prev=2. curr=3.

curr=3: nxt=4. 3.next=2. prev=3. curr=4.

curr=4: nxt=5. 4.next=3. prev=4. curr=5.

curr=5: nxt=None. 5.next=4. prev=5. curr=None. Stop.

Return prev=5. 5->4->3->2->1 ok

Recursive: reverseList(5) returns 5. Then 4.next.next=4 -> 5->4->...->1. Each call reverses one link.

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one pass	three pointer variables

Recursive solution is elegant but O(n) stack space — mention this trade-off.

Follow-up: Reverse Linked List II (#92) — reverse between positions l and r.

Follow-up: Reverse Nodes in k-Group (#25) — apply this logic to groups of k.

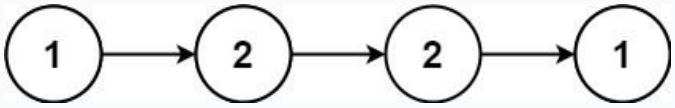
EASY | Linked List | LeetCode #234 | 6 of 9 | 6/9

Palindrome Linked List

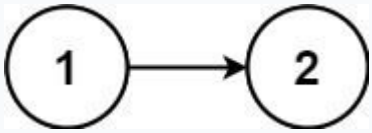
QUESTION

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1	
Input	`head = [1,2,2,1]`
Output	`true`



Example 2	
Input	`head = [1,2]`
Output	`false`



CONSTRAINTS

- The number of nodes in the list is in the range [1, 10^5].
- 0 <= Node.val <= 9
- Follow up: Could you do it in O(n) time and O(1) space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We need to check if the linked list reads the same forwards and backwards.

We can only traverse with next pointers — no random access.

The follow-up asks for O(n) time and O(1) space. Right?"

"A few quick questions:

Can the list be empty or have one node? Yes — trivially palindromes.

Should we restore the list after checking? Good practice; the problem doesn't require it."

"Let me walk the example: [1,2,2,1] -> palindrome ok. [1,2] -> not ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = the list copied into an array
return values == reverse(values)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but copies the list. Reverse the second half in place and compare it with the first half (restore it afterwards): O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra array.

We can do this in O(1) extra space with three steps:

1. Find the midpoint using slow/fast pointers.
2. Reverse the second half in-place.
3. Compare first half with reversed second half node by node.

Pseudocode:

slow, fast = head, head

```
while fast and fast.next: slow = slow.next; fast =
  fast.next.next
second = reverse(slow)
```

left, right = head, second

```
while right:
if left.val != right.val: return False
left = left.next; right = right.next
return True
```

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _234_PalindromeLinkedList:
    def isPalindrome(self, head):
        def reverse(node):
            prev, cur = None, node
            while cur:
                nxt = cur.next; cur.next = prev; prev, cur
 = cur, nxt
            return prev

        slow = fast = head
        while fast and fast.next:
            slow = slow.next; fast = fast.next.next

        second = reverse(slow)
        left, right = head, second
        while right:
            if left.val != right.val:
                return False
            left = left.next; right = right.next
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace isPalindrome(1->2->2->1)."

Find middle: slow/fast. slow=2(second 2), fast=None.

Reverse second half from slow: 2->2->1 reversed -> 1->2->2. Wait: slow is mid (len//2).

n=4: after slow/fast: slow=node(2) at index 2. Reverse from slow: 1->2->(stopped).

Compare: l1=head=1, l2=1. Match. l1=2, l2=2. Match. l1=2, l2=None? Hmm.

Correct: slow ends at index 2, reverse [index 2..end] = [2,1]->[1,2]. Then compare [1,2] vs [1,2] ok

1->2->3->2->1: mid=3. Reverse [3,2,1]->[1,2,3]. Compare [1,2,?] vs [1,2,3]: stop at mid. ok

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
find mid O(n/2) + reverse O(n/2) + compare O(n/2)	constant extra pointers

Trade-off: the list is modified during checking. To restore: reverse again after comparison.

Follow-up: Valid Palindrome (#125) — same idea on a string, ignoring non-alphanumeric.

Follow-up: Palindrome Number (#9) — similar reverse-half technique on an integer.

EASY | Linked List | LeetCode #706 | 7 of 9 | 7/9

Design HashMap

QUESTION

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

CONSTRAINTS

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.
- 0 <= key, value <= 10^6
- At most 10^4 calls will be made to put, get, and remove.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We need to implement a HashMap from scratch — no built-in dict or hash libraries.

Support put(key, val), get(key) -> -1 if missing, and remove(key).

Keys and values are non-negative integers up to 10^6. Right?"

"A few quick questions:

At most 10^4 calls total — so a simple fixed-size array could work.

Does put update the value if the key exists? Yes.

Does remove need to return anything? No."

"Let me walk the example: put(1,1), put(2,2), get(1)->1, get(3)->-1, remove(2), get(2)->-1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep an array with one slot for every possible key (0
    to 10^6)
put, get, remove read or write that slot
```

TIME	SPACE
O(1)	O(10^6)

WHY IT IS SLOW

Constant time but memory for every possible key. Buckets of (key, value) pairs chosen by key modulo a prime size give O(n) space and O(1) average operations.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) linear scan per operation.

Use an array of buckets with separate chaining: each bucket holds a small list.

Hash function: key % num_buckets. We pick a large prime (e.g. 1009) to spread keys evenly.

Average O(1) per operation when the load factor is low.

Pseudocode:

__init__: buckets = [[] for _ in range(1009)]

put: idx = key % 1009; update existing pair or append

get: scan bucket for key, return value or -1

remove: filter key out of bucket

Time: O(1) average. Space: O(n + 1009)."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _706_DesignHashMap:
    def __init__(self):
        self._size = 1009
        self._buckets = [[] for _ in range(self._size)]

    def _idx(self, key):
        return key % self._size

    def put(self, key, value):
        bucket = self._buckets[self._idx(key)]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))

    def get(self, key):
        for k, v in self._buckets[self._idx(key)]:
            if k == key:
                return v
        return -1

    def remove(self, key):
        idx = self._idx(key)
        self._buckets[idx] = [(k, v) for k, v in self._buc
kets[idx] if k != key]
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

```
put(1,1): bucket[1]=[(1,1)]. put(2,2):
  bucket[2]=[(2,2)].
get(1): bucket[1] -> (1,1) -> 1 ok. get(3):
  bucket[3]=[] -> -1 ok.
put(2,1): bucket[2] -> update -> [(2,1)].
remove(2): bucket[2]=[]. get(2): [] -> -1 ok.
```

PHASE 5 TEST & DEBUG

Prime bucket count minimises collisions for common key distributions

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) average; O(n/B) worst case if all keys hash to same bucket (B = bucket count)	**O(n + B)** n stored pairs + B bucket slots

Trade-off: larger B reduces collisions but wastes memory for sparse maps.

Follow-up: Design HashSet (#705) — identical structure, store keys only.

Follow-up: open addressing (linear probing) is an alternative — avoids linked lists but needs tombstones.

EASY | Linked List | LeetCode #876 | 8 of 9 | 8/9

Middle of the Linked List

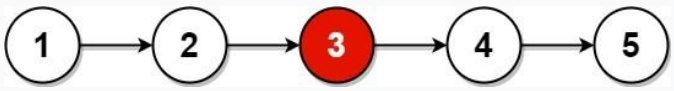
QUESTION

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

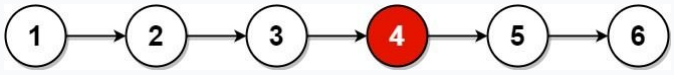
Example 1

Input	`head = [1,2,3,4,5]`
Output	`[3,4,5]`
Explanation	*The middle node of the list is node 3.*



Example 2

Input	`head = [1,2,3,4,5,6]`
Output	`[4,5,6]`
Explanation	*Since the list has two middle nodes with values 3 and 4, we return the second one.*



CONSTRAINTS

- The number of nodes in the list is in the range [1, 100].
- 1 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the middle node of a singly linked list.

If there are two middle nodes (even length), return the SECOND one.

Right?"

"A few quick questions:

Can the list be empty? No — length is at least 1.

Return the node itself, not its value? Yes."

"Let me walk the examples:

[1,2,3,4,5]: length 5 -> middle index 2 -> node(3) ok

[1,2,3,4,5,6]: two middles at 2,3 -> return second -> node(4) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count the length by walking the list
walk length / 2 steps from the head and return that
    node
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(1)

NOTE

Not slow: two passes of O(n). A slow pointer moving one step and a fast pointer moving two finds the middle in a single pass.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is doing two full passes — we can find the middle in one pass.

Slow/fast pointer: slow moves 1 step, fast moves 2.

When fast reaches the end, slow is at the middle.

For even-length lists, slow naturally lands on the SECOND middle (desired behaviour).

Pseudocode:

```
slow = fast = head
while fast and fast.next:
slow = slow.next; fast = fast.next.next
return slow
```

Time: O(n). Space: O(1).

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _876_MiddleLinkedList:
    def middleNode(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

[1,2,3,4,5]:

s=1,f=1 -> s=2,f=3 -> s=3,f=5 -> fast.next=None -> stop. return node(3) ok

[1,2,3,4,5,6]:

s=1,f=1 -> s=2,f=3 -> s=3,f=5 -> s=4,f=None(6.next) -> stop. return node(4) ok

[1]:

fast.next=None immediately -> return node(1) ok

PHASE 5 TEST & DEBUG

The 'fast and fast.next' guard handles odd and even lengths uniformly

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
fast covers the list in n/2 iterations	two pointers

To get the FIRST middle for even-length, use 'while fast.next and fast.next.next' instead.

This building-block appears in Palindrome Linked List (#234) and Reorder List (#143).

Follow-up: Kth Node From End (#19) — two-pointer gap of k instead of 2:1 speed ratio.

IN YOUR APP | Expose | Editorial | Community

EASY | Linked List | LeetCode #1474 | 9 of 9 | 9/9

Delete N Nodes After M Nodes of Linked List

QUESTION

You are given the head of a linked list and two integers m and n.

Traverse the linked list and remove some nodes in the following way:

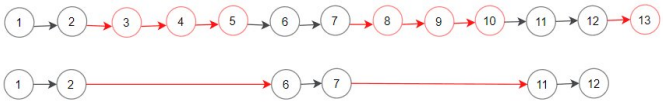
CONSTRAINTS

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

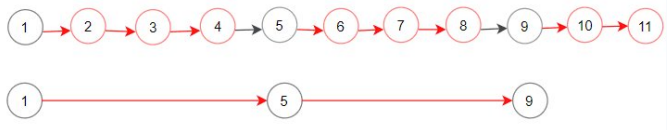
Example 1

Input	`head = [1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2, n = 3`
Output	`[1,2,6,7,11,12]`
Explanation	*Keep the first (m = 2) nodes starting from the head of the linked List (1 ->2) show in black nodes.* *Delete the next (n = 3) nodes (3 -> 4 -> 5) show in read nodes.* *Continue with the same procedure until reaching the tail of the Linked List.* *Head of the linked list after removing nodes is returned.*



Example 2

Input	`head = [1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3`
Output	`[1,5,9]`
Explanation	*Head of linked list after removing nodes is returned.*



- The number of nodes in the list is in the range [1, 10^4].
- 1 <= Node.val <= 10^6
- 1 <= m, n <= 1000
- Follow up: Could you solve this problem by modifying the list in-place?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Walk the list in cycles: keep the first m nodes, delete the next n nodes, repeat.

Do this until the end of the list and return the modified head. Right?"

"A few quick questions:

m and n are guaranteed to be at least 1? Yes.

Modify in-place — no new nodes? Yes, just rewire pointers."

"Let me walk the example: [1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3.

Keep 1,2 -> delete 3,4,5 -> keep 6,7 -> delete 8,9,10 -> keep 11,12 -> delete 13.

-> [1,2,6,7,11,12] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
put all nodes in an array
keep m nodes, skip n nodes, repeat
relink the kept nodes from the array
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but stores every node. Walking the list with a pointer, skipping n nodes after every m and relinking in place, uses O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) auxiliary list.

Walk m steps forward (keeping nodes), then walk n steps forward (skipping them),

rewire the tail of the kept section directly to the node after the skipped block.

Repeat until we run out of nodes.

Pseudocode:

```
cur = head
while cur:
```

walk m-1 steps (cur is now tail of kept section)

walk n steps past the nodes to delete

```
cur.next = the node after the deleted block
cur = cur.next
return head
```

Time: O(n_total). Space: O(1).

Does that make sense before I code it?"

PHASE 3 OPTIMIZE

Key insight: advancing m steps then skipping n steps with pointer manipulation processes the whole list in one pass — no auxiliary array, O(1) extra space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1474_DeleteNNodesAfterM:
    def deleteNodes(self, head, m, n):
        cur = head
        while cur:
            for _ in range(m - 1):
                if cur: cur = cur.next
            skip = cur.next if cur else None
            for _ in range(n):
                if skip: skip = skip.next
            if cur:
                cur.next = skip
                cur = skip
        return head
```

PHASE 5 TEST & DEBUG

"Let me trace the pointer rewiring steps and show how the list structure stays consistent."

[1,2,3,4,5,6,7], m=2, n=3:

cur=1. walk m-1=1 step -> cur=2. skip=3, walk n=3 -> skip=6. 2.next=6. cur=6.

cur=6. walk 1 -> cur=7. skip=None (7.next). walk 3 (skip stays None). 7.next=None. cur=None.

-> [1,2,6,7] ok

[1], m=1, n=1:

cur=1. walk 0 steps -> cur=1. skip=None, walk 1 -> skip=None. 1.next=None. cur=None.

-> [1] ok

Guards prevent crashing on short lists

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(total_nodes)	**O(1)**
each node visited exactly once	only pointer variables

Contrast with the brute that uses O(n) auxiliary storage.

Follow-up: what if m and n change each cycle? Accept arrays of m and n values.

Follow-up: Remove Duplicates from Sorted List (#83) — same rewire pattern, different condition.